09 - Operadores

Operadores aritméticos en Python

Los operadores aritméticos nos permiten realizar operaciones matemáticas sobre números. Los operadores aritméticos básicos en Python son los son:

Operador
+ Suma: Suma dos valores
- Resta: Resta el segundo valor del primero
* Multiplicación: Multiplica dos valores
/ División: Divide el primer valor por el segundo (retorna un float)
% Módulo: Retorna el residuo de la división del primer valor por el segundo
** Potencia: Eleva el primer valor a la potencia del segundo
// División entera: Divide el primer valor por el segundo y retorna la parte entera

Lista de operadores aritméticos

Suma +

El operador de suma (+) se utiliza para sumar dos números. También se puede utilizar para concatenar cadenas y listas en Python.

```
a = 5
b = 3
resultado = a + b  # resultado es 8
```

Resta -

El operador de resta (-) se utiliza para restar un número de otro.

09 - Operadores

```
a = 10
b = 7
resultado = a - b  # resultado es 3
```

Multiplicación *

El operador de multiplicación (*) se utiliza para multiplicar dos números.

```
a = 5
b = 4
resultado = a * b  # resultado es 20
```

División /

El operador de división (/) se utiliza para dividir un número entre otro. En Python 3, la división siempre devuelve un número de punto flotante (float).

```
a = 10
b = 2
resultado = a / b  # resultado es 5.0
```

División Entera //

El operador de división entera (//) se utiliza para dividir un número entre otro y devolver el cociente entero, descartando cualquier parte decimal.

```
a = 10
b = 3
```

```
resultado = a // b  # resultado es 3
```

Módulo %

El operador de módulo (%) se utiliza para obtener el residuo de una división entre dos números.

```
a = 10
b = 3
resultado = a % b  # resultado es 1
```

Potencia **

El operador de potencia (**) se utiliza para elevar un número a una potencia.

```
a = 2
b = 3
resultado = a ** b  # resultado es 8
```

09 - Operadores

Operadores de asignación en Python

Los operadores de asignación en Python nos permiten asignar valores a variables.

El operador de asignación más común es el operador =. Este operador se utiliza para asignar el valor de la derecha a la variable de la izquierda.

Además del operador de asignación básico (=), existen operadores de asignación compuestos que combinan una operación aritmética con la asignación, simplificando y optimizando el código.

Operador de asignación básico =

El operador de asignación básico (=) se utiliza para asignar un valor a una variable. Por ejemplo:

```
numero = 10
```

En este ejemplo, **numero** es una variable a la que se le asigna el valor 10.

El operador de asignación también se utiliza para actualizar el valor de una variable. Por ejemplo:

```
a = 5 # Asignación inicial
a = 10 # Actualización del valor
```

09 - Operadores

Operadores de asignación compuestos

Operador +=

El operador **+=** se utiliza para sumar el valor de la derecha a la variable y asignar el resultado a la variable.

```
contador = 5
contador += 3  # Equivalente a contador = contador + 3
```

Después de esta operación, **contador** será igual a 8.

Operador -=

El operador **-=** se utiliza para restar el valor de la derecha a la variable y asignar el resultado a la variable.

```
total = 100
descuento = 20
total -= descuento  # Equivalente a total = total - descuento
```

Después de esta operación, **total** será igual a 80.

Operador *= y /=

Estos operadores se utilizan para multiplicar (*=) y dividir (/=) la variable por el valor de la derecha y asignar el resultado a la variable, respectivamente.

```
cantidad = 5
```

09 - Operadores

```
cantidad *= 2  # Equivalente a cantidad = cantidad * 2

precio = 100
descuento = 20
precio /= (100 - descuento) / 100  # Equivalente a precio = precio
/ ((100 - descuento) / 100)
```

Operador //=

El operador **//=** se utiliza para realizar una división entera y asignar el resultado a la variable.

```
numero = 25
numero //= 4  # Equivalente a numero = numero // 4, el resultado
es 6
```

Después de esta operación, **numero** será igual a 6.

Operador %= y **=

Estos operadores se utilizan para calcular el módulo (%=) y la potencia (**=) de la variable y asignar el resultado a la variable, respectivamente.

```
numero = 10
numero %= 3  # Equivalente a numero = numero % 3

base = 2
exponente = 3
base **= exponente  # Equivalente a base = base ** exponente
```

09 - Operadores

Operador %= y **=

Estos operadores se utilizan para calcular el módulo (%=) y la potencia (**=) de la variable y asignar el resultado a la variable, respectivamente.

```
numero = 10
numero %= 3  # Equivalente a numero = numero % 3

base = 2
exponente = 3
base **= exponente  # Equivalente a base = base ** exponente
```

Operadores de comparación

Los operadores de comparación en Python nos permiten comparar dos valores y determinar si son iguales, diferentes, mayores o menores:

Los resultados de las comparaciones son valores booleanos (True o False), y son fundamentales para la lógica de control de flujo.

Operador	Nombre	Descripción
==	Igual a	Compara si dos valores son iguales
!=	Diferente de	Compara si dos valores son diferentes
>	Mayor que	Verifica si un valor es mayor que otro
<	Menor que	Verifica si un valor es menor que otro
>=	Mayor o igual que	Verifica si un valor es mayor o igual
<=	Menor o igual que	Verifica si un valor es menor o igual

09 - Operadores

Lista de operadores de comparación

Igualdad ==

El operador de igualdad (==) se utiliza para verificar si dos valores son iguales.

```
a = 5
b = 5
es_igual = (a == b)  # True, ya que a y b son iguales
```

Desigualdad !=

El operador de desigualdad (!=) se utiliza para verificar si dos valores no son iguales.

```
a = 5
b = 3
no_es_igual = (a != b)  # True, ya que a no es igual a b
```

Mayor que >

El operador mayor que (>) se utiliza para verificar si un valor es mayor que otro.

```
a = 10
```

```
b = 5
es_mayor = (a > b)  # True, ya que a es mayor que b
```

Menor que <

El operador menor que (<) se utiliza para verificar si un valor es menor que otro.

```
a = 3
b = 7
es_menor = (a < b)  # True, ya que a es menor que b
```

Mayor o igual que >=

El operador mayor o igual que (>=) se utiliza para verificar si un valor es mayor o igual que otro.

```
a = 10
b = 10
es_mayor_igual = (a >= b)  # True, ya que a es igual a b
```

Menor o igual que <=

El operador menor o igual que (<=) se utiliza para verificar si un valor es menor o igual que otro.

```
a = 5
b = 7
es_menor_igual = (a <= b)  # True, ya que a es menor que b
```

Operadores de identidad

Los operadores de identidad en Python nos permiten verificar si dos variables se refieren al mismo objeto en la memoria. Estos operadores son útiles para comparar la identidad de objetos (es decir, si dos variables apuntan al mismo lugar en la memoria).

Los operadores is y is not son muy eficientes ya que simplemente comparan las direcciones de memoria de dos objetos (no realizan ninguna operación adicional).

Operador	**Descripción**
is	Verifica si dos variables apuntan al mismo objeto
is not	Verifica si dos variables no apuntan al mismo objeto

Lista de operadores de identidad

Operador is

El operador is se utiliza para verificar si dos variables se refieren al mismo objeto en la memoria.

```
a = [1, 2, 3]
b = a
es_igual = (a is b)  # True, ya que a y b se refieren al mismo
objeto
```

En este ejemplo, tanto **a** como **b** apuntan al mismo objeto en la memoria, por lo que la expresión a is **b** es **True**.

09 - Operadores

Operador is not

El operador **is not** se utiliza para verificar si dos variables NO se refieren al mismo objeto en la memoria.

```
x = 10
y = 10
no_es_igual = (x is not y)  # False, ya que x e y se refieren al
mismo objeto
```

En este ejemplo, tanto **x** como **y** apuntan al mismo objeto en la memoria (en Python, los enteros pequeños se almacenan en la misma ubicación), por lo que la expresión **x is not y** es **False**.

Ejemplos de uso

Verificación de identidad de variables

```
x = 100
y = x

son_iguales = (x is y)  # True, ya que x e y son el mismo objeto
en memoria
```

Verificación de identidad de listas

```
lista1 = [1, 2, 3]
lista2 = [1, 2, 3]
lista3 = lista1

son_iguales = (lista1 is lista2)  # False, ya que lista1 y lista2
no son el mismo objeto
son_iguales2 = (lista1 is lista3)  # True, ya que lista1 y lista3
```

```
son el mismo objeto
```

Verificación de identidad de objetos

```
class MiClase:
    pass

objeto1 = MiClase()
objeto2 = MiClase()
objeto3 = objeto1

son_iguales = (objeto1 is objeto2)  # False, ya que objeto1 y
objeto2 no son el mismo objeto
son_iguales2 = (objeto1 is objeto3)  # True, ya que objeto1 y
objeto3 son el mismo objeto
```

09 - Operadores

Operadores lógicos

Los operadores lógicos en Python nos permiten combinar condiciones booleanas para formar expresiones más complejas.

Operador	Descripción
and	Retorna True si ambos operandos son True
or	Retorna True si al menos uno de los operandos es True
not	Retorna True si el operando es False

Lista de operadores lógicos

Operador and

El operador and se utiliza para combinar dos condiciones, y la expresión resultante será True solo si ambas condiciones son True.

```
a = 5
b = 10
c = 15

resultado = (a < b) and (b < c)  # True, ya que ambas condiciones son True
```

Operador or

El operador or se utiliza para combinar dos condiciones, y la expresión resultante será True si al menos una de las condiciones es True.

09 - Operadores

```
a = 5
b = 10
c = 3

resultado = (a > b) or (b < c)  # False, ya que ninguna de las
condiciones es True
```

Operador not

El operador not se utiliza para negar una condición, es decir, convierte True en False y viceversa.

```
verdadero = True
falso = not verdadero  # falso es False

numero = 5
es_mayor_a_10 = numero > 10
no_es_mayor_a_10 = not es_mayor_a_10
```

Ejemplos de uso

Combinación con and

```
edad = 25
tiene_licencia = True
puede_conducir = (edad >= 18) and tiene_licencia  # True
```

Combinación con or

09 - Operadores

```
esta_lloviendo = False
es_frio = True

quiere_quedarse_en_casa = esta_lloviendo or es_frio  # True
```

Negación con not

```
es_dia = True
es_noche = not es_dia  # False
```

Combinación de operadores lógicos

Es posible combinar varios operadores lógicos en una sola expresión para crear condiciones más complejas.

```
numero = 15

es_multiplo_de_3 = (numero % 3 == 0)
es_multiplo_de_5 = (numero % 5 == 0)

es_multiplo_de_3_o_5 = es_multiplo_de_3 or es_multiplo_de_5
# True, ya que 15 es múltiplo de 3

es_multiplo_de_3_y_5 = es_multiplo_de_3 and es_multiplo_de_5
# True, ya que 15 es múltiplo de 3 y 5

no_es_multiplo_de_3 = not es_multiplo_de_3  # False
```

09 - Operadores

Precedencia de operadores lógicos

Cuando se combinan múltiples operadores lógicos en una expresión, es importante entender su precedencia para evaluar correctamente la expresión.

La precedencia de operadores lógicos en Python sigue este orden:

- not
- and
- or

Por lo tanto, en una expresión con **and**, **and** se evalúa antes que **or**.

Sin embargo, no conviene abusar de esto a la hora de escribir nuestro código. No os importa usar paréntesis si mejora la legibilidad.

Operadores de pertenencia

Los operadores de pertenencia en Python nos permiten verificar si un valor está presente dentro de una secuencia (como una cadena, lista, tupla, o un conjunto).

Operador	Descripción
in	Retorna True si el valor se encuentra en la secuencia
not in	Retorna True si el valor no se encuentra en la secuencia

09 - Operadores

Lista de operadores de pertenencia

Operador in

El operador in se utiliza para verificar si un valor está presente en una secuencia.

```
frutas = ["manzana", "banana", "cereza"]

es_banana = "banana" in frutas  # True
es_uva = "uva" in frutas  # False
```

En este ejemplo, estamos verificando si "banana" está presente en la lista frutas, lo cual es True. También verificamos si "uva" está presente, lo cual es False.

Operador not in

El operador not in se utiliza para verificar si un valor NO está presente en una secuencia.

```
frutas = ["manzana", "banana", "cereza"]

no_es_uva = "uva" not in frutas  # True
no_es_manzana = "manzana" not in frutas  # False
```

En este ejemplo, estamos verificando si "uva" NO está presente en la lista frutas, lo cual es True. También verificamos si "manzana" NO está presente, lo cual es False.

09 - Operadores

Ejemplos de uso

Verificación de elementos en una lista

```
colores = ["rojo", "verde", "azul"]

es_azul = "azul" in colores  # True
no_es_amarillo = "amarillo" not in colores  # True
```

Verificación de caracteres en una cadena

```
nombre = "Luis"

tiene_a = "a" in nombre  # True
no_tiene_z = "z" not in nombre  # True
```

Verificación de elementos en un conjunto

```
vocales = {"a", "e", "i", "o", "u"}

es_a_vocal = "a" in vocales  # True
no_es_y = "y" not in vocales  # True
```

Operadores bitwise

Los operadores bitwise en Python permiten manipular los bits individuales de números enteros. Son útiles para realizar operaciones a nivel de bits, como desplazamiento, **AND**, **OR**, y complemento.

Los operadores bitwise en Python son los siguientes:

09 - Operadores

Operador	Descripción
AND (&)	Realiza una operación AND bit a bit
OR (|)	Realiza una operación OR bit a bit
XOR (^)	Realiza una operación XOR bit a bit
Desplazamiento Izquierda (<<)	Desplaza los bits a la izquierda
Desplazamiento Derecha (>>)	Desplaza los bits a la derecha
Complemento (~)	Devuelve el complemento a uno del número

Operador AND &

El operador AND (&) compara cada bit de dos operandos y devuelve 1 si ambos bits son 1, de lo contrario devuelve 0.

```
# Ejemplo de operador AND
a = 10  # 1010 en binario
b = 3   # 0011 en binario
resultado = a & b
print(resultado)  # Salida: 2
# Explicación: 1010 & 0011 = 0010 (decimal 2)
```

Operador OR |

El operador OR (|) compara cada bit de dos operandos y devuelve 1 si al menos uno de los bits es 1.

```
# Ejemplo de operador OR
a = 10  # 1010 en binario
b = 3   # 0011 en binario
resultado = a | b
print(resultado)  # Salida: 11
# Explicación: 1010 | 0011 = 1011 (decimal 11)
```

09 - Operadores

Operador XOR ^

El operador XOR (^) compara cada bit de dos operandos y devuelve 1 si exactamente uno de los bits es 1, pero no ambos.

```
# Ejemplo de operador XOR
a = 10  # 1010 en binario
b = 3   # 0011 en binario
resultado = a ^ b
print(resultado)  # Salida: 9
# Explicación: 1010 ^ 0011 = 1001 (decimal 9)
```

Operadores de desplazamiento

Los operadores de desplazamiento mueven los bits de un número hacia la izquierda **(<<)** o hacia la derecha **(>>)** en la cantidad especificada.

Ejemplo de Desplazamiento Izquierda

```
# Ejemplo de desplazamiento izquierda
a = 5   # 101 en binario
resultado = a << 2
print(resultado)  # Salida: 20
# Explicación: 101 << 2 = 10100 (decimal 20)
```

Ejemplo de Desplazamiento Derecha

```
# Ejemplo de desplazamiento derecha
a = 20   # 10100 en binario
resultado = a >> 2
print(resultado)  # Salida: 5
```

```
# Explicación: 10100 >> 2 = 101 (decimal 5)
```

Operador de complemento ~

El operador de complemento **(~)** invierte todos los bits de un número.

```
# Ejemplo de operador de complemento
a = 5   # 101 en binario
resultado = ~a
print(resultado)  # Salida: -6
# Explicación: ~101 = -110 (en complemento a dos, -6 en decimal)
```

La expresión ~x es equivalente a la fórmula matemática de complemento a uno: -(x + 1).
En Python, ~5 da como resultado -6. En la representación binaria, si 5 es 0101, ~5 sería 1010, que en complemento a dos es -6.